SQL Quest — Installation Guide

SQL Quest · Installation Guide · 2026-11-05

This guide is for whoever puts SQL Quest online (an IT technician, lab administrator or teacher comfortable with servers).

1. Choose a version

	Server version (recommended)	**Browser-only version**
What you upload	The whole repository, run with Node.js + PostgreSQL	Only the `sql-quest/` folder, to any static web host
Where data is stored	Your PostgreSQL database	Each visitor's browser
Students on several devices, one teacher dashboard	✅	❌ each browser is separate
Answers hidden and marking done on the server	✅ cheat-resistant	❌ answers are in the page code
Cost and effort	Needs a small server or hosting plan with PostgreSQL	Free static hosting; no setup
Best for	Real classes and anything counting towards grades	Demos, self-study, a single lab computer

Both versions use the same game files. The page detects automatically which version it is running in.

---

2. Browser-only version (static hosting)

1. Upload the **contents** of the `sql-quest/` folder (or unzip `sql-quest-upload.zip`) to your host: Netlify, GitHub Pages, cPanel `public_html`, and so on. Keep the sub-folders.
2. Open the site. Demo accounts are created in each browser: `admin/admin123`, `teacher/teacher123`, `student/student123`.
3. Log in as admin → **Settings** → untick **Show demo accounts**, then change the admin password.

The site must be opened over `http(s)://`, not by double-clicking `index.html`. To try it on your own computer: `cd sql-quest && python3 -m http.server 8000`, then open http://localhost:8000.

---

3. Server version with Docker (easiest)

You need a machine with Docker and Docker Compose: a school server, a cloud VM or a desktop.

```
git clone <your-repository-url> sql-quest
cd sql-quest
cp .env.example .env
```

Edit `.env` and set at least:

```
DB_PASSWORD=a-long-random-database-password
ADMIN_PASSWORD=a-strong-admin-password
```

Start it:

```
docker compose up -d --build
```

Open `http://<server-address>:3000` and log in as `admin` with your `ADMIN_PASSWORD`. Then follow the Administrator Guide, section 2.

Useful commands:

```
docker compose logs -f app        # view server logs
docker compose restart app        # restart
docker compose down               # stop (data is kept in the "pgdata" volume)
```

To **try it with sample data**, set `SEED_DEMO=true` in `.env` **before the first start** (on an empty database). This creates the demo accounts, classes and progress. Don't use demo data on a real site.

---

4. Server version without Docker

Requirements: **Node.js 18 or newer** and **PostgreSQL 13 or newer**.

1. Create a database and user:

   ```
   CREATE USER sqlquest WITH PASSWORD 'choose-a-password';
   CREATE DATABASE sqlquest OWNER sqlquest;
   ```

2. Install and start:

   ```
   cd server
   npm ci --omit=dev
   DATABASE_URL=postgres://sqlquest:choose-a-password@localhost:5432/sqlquest \
   ADMIN_PASSWORD='a-strong-admin-password' \
   npm start
   ```

The tables are created automatically on first start. Keep the process running with a service manager (systemd, pm2, or your host's process manager).

3. Open `http://<server-address>:3000`.

Hosting platforms (Render, Railway, Fly.io, and similar)

1. Create a **PostgreSQL** database on the platform and copy its connection URL.
2. Create a **web service** from this repository:
   - **Build command:** `cd server && npm ci --omit=dev`
   - **Start command:** `node server/src/index.js`
   - Or let the platform use the included `Dockerfile`.
3. Set the environment variables `DATABASE_URL`, `ADMIN_PASSWORD`, `COOKIE_SECURE=true`, `TRUST_PROXY=true`, and `PGSSL=true` if the database requires SSL.
4. Run **one instance only** (see section 7).

---

5. Settings (environment variables)

Variable	Required	Default	Purpose
`DATABASE_URL`	✅	—	PostgreSQL connection string.
`ADMIN_PASSWORD`	on first start	random (printed in the log)	Password for the first admin account, created when the database is empty.
`ADMIN_USERNAME`		`admin`	Username for that account.
`PORT`		`3000`	Port the server listens on.
`SEED_DEMO`		`false`	`true` = load demo accounts and sample data into an empty database.
`ALLOW_DEMO_RESET`		`false`	`true` = show the admin "Reset platform to demo data" button.
`COOKIE_SECURE`		auto	`true` when served over HTTPS (needed behind most proxies).
`TRUST_PROXY`		—	`true` when behind a reverse proxy or load balancer (Nginx, Render, Railway…).
`PGSSL`		`false`	`true` to connect to PostgreSQL over SSL (most hosted databases).
`SESSION_DAYS`		`30`	How long users stay logged in.
`SANDBOX_WORKERS`		CPUs (max 4)	Number of parallel SQL sandboxes for marking.

HTTPS

Put the server behind HTTPS before real use, for example a reverse proxy such as Nginx or Caddy, or your hosting platform's built-in HTTPS. Then set `COOKIE_SECURE=true` and `TRUST_PROXY=true`.

---

6. Backups and updates

Database backups, for example nightly:

```
# Docker
docker compose exec db pg_dump -U sqlquest sqlquest > sqlquest-$(date +%F).sql
# Without Docker
pg_dump "$DATABASE_URL" > sqlquest-$(date +%F).sql
```

Restore into an empty database with `psql "$DATABASE_URL" < backup.sql`, with the app stopped. Admins can also download a JSON backup from **Settings → Export backup**.

Updating to a new version:

```
git pull
docker compose up -d --build     # or: cd server && npm ci --omit=dev, then restart the
process
```

The database schema is created or updated automatically at start-up.

Emergency admin password reset: stop the app, then run `DATABASE_URL=... node server/scripts/reset-password.js <username> <new-password> --make-admin`, and start the app again.

---

7. How it works, and its limits

- The server keeps the working data in memory and writes every change to PostgreSQL straight away, inside a transaction. Because of this, **run exactly one server process per database**. One process comfortably serves a school (hundreds of students working at the same time).
- Student queries run in isolated worker threads, each with its own throwaway SQLite database, a time limit, a memory cap and a row cap. They never touch PostgreSQL.
- The game rules (`sql-quest/js/actions.js`, `engine.js`, `grader.js`) are the same code in the browser and on the server; the server's copy is the one that counts.
- PostgreSQL tables: `users`, `classes`, `progress`, `submissions`, `badges`, `user_badges`, `assignments`, `teams`, `reviews`, `custom_challenges`, `custom_lessons`, `hidden_content`, `drafts`, `settings`, `sessions`.

Running the automated tests

```
cd server
npm install
TEST_DATABASE_URL=postgres://user:pass@localhost:5432/sqlquest_test npm test
```

⚠️ The test database is **wiped** at the start of each run. Never point it at your real database.